

数据结构与算法

张史梁

2026-9

课程内容

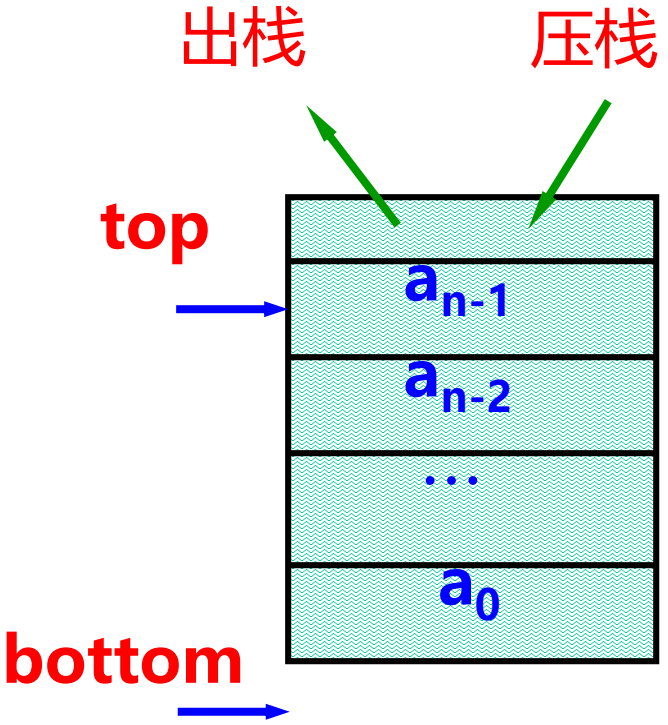
- 栈
- 队列

1. 典型的线性结构
2. 是操作受限的线性结构

- 栈和队列的应用和讨论

3.1 栈

- 限制在一端访问的线性表
- 后进先出(Last-In First-Out, LIFO), 也称“下推表”
- ‘压栈’(push)与出栈（pop）
- ‘栈顶’ (top) 与 ‘栈底’ (bottom)

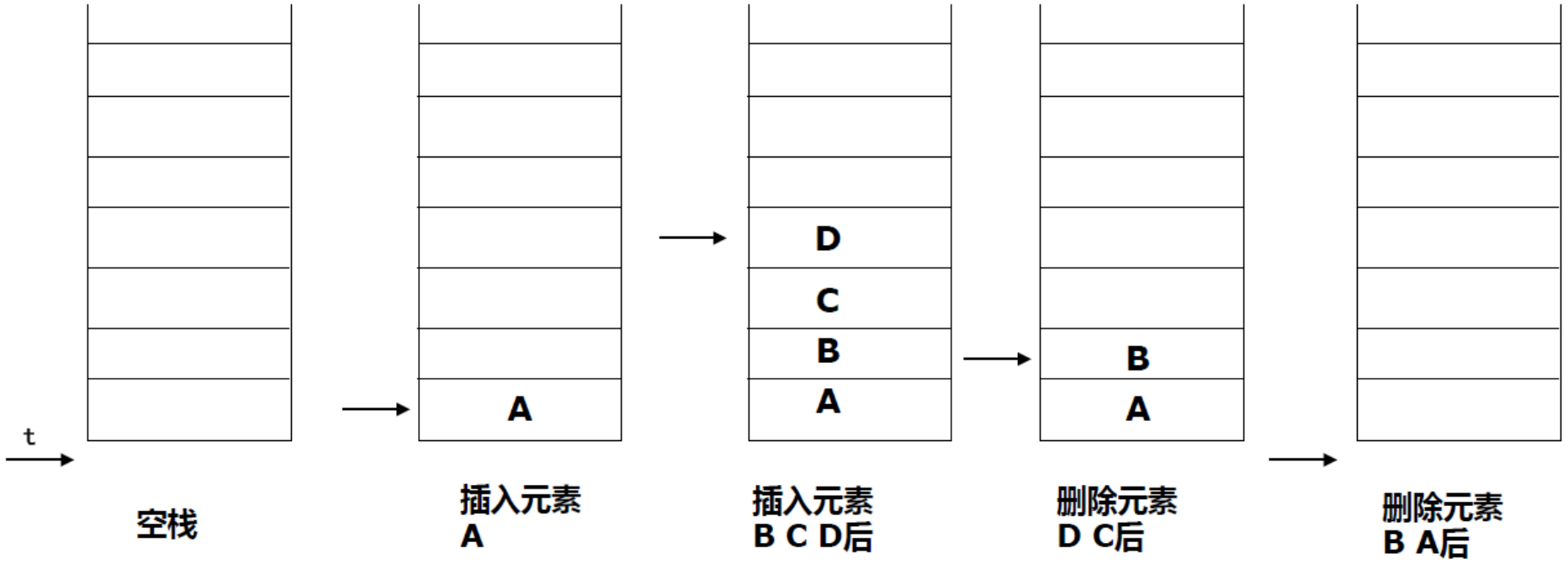




栈的主要操作

- 入栈（ push ）
- 出栈（pop）
- 取栈顶元素（ top ）
- 判断栈是否为空（ isEmpty ）
- 判断栈是否为满（ isFull ）

栈的插入、删除





思考

- 如果入栈的顺序为1，2，3，4，则出栈的顺序可以有那些?
  - 1，2，3，4
  - 1，2，4，3
  - 1，3，2，4
  - 1，3，4，2
  - 1，4，2，3
  - 2，1，3，4
  - 。。。



3.1.1 栈的ADT

```
template <class T>                    // 栈的元素类型为 T
class Stack {
public:                               // 栈的运算集
    void clear();                     // 变为空栈
    bool push(const T item);          // item入栈,成功则返回真,否则返回假
    bool pop(T & item);               // 返回栈顶内容并弹出,成功返回真,否则返回假
    bool top(T& item);                // 返回栈顶内容但不弹出,成功返回真,否则返回假
    bool isEmpty();                   // 若栈已空返回真
    bool isFull();                    // 若栈已满返回真
};
```

- 思考题
  - 给定一个入栈序列，和一个出栈序列，请你写出一个程序，判定出栈序列是否合法?
  - 给定一个入栈序列，序列长度为N，请计算由多少种出栈序列?

1. 按入栈序列依次将元素压入栈
2. 每压入一个元素后，检查栈顶是否等于出栈序列当前待出栈的元素
3. 如果相等，就弹出，并移动出栈序列指针
4. 重复上述过程，直到所有入栈元素处理完
5. 最后若栈为空，则出栈序列合法；否则不合法

栈的实现方式

- 顺序栈（array-based stack）
  - 使用向量实现，本质上是顺序表的简化
  - 关键是确定哪一端作为栈顶
  - 上溢、下溢问题

- 链式栈（linked stack）
  - 用单链表方式存储，其中指针的方向是从栈顶向下链接



3.1.2 顺序栈

```
template <class T> class arrStack : public Stack <T> {
    private:                                        // 栈的顺序存储
        int         mSize;                          // 栈中最多可存放的元素个数
        int         top;                            // 栈顶位置，应小于mSize
        T           *st;                            // 存放栈元素的数组
    public:                                         // 栈的运算的顺序实现
        arrStack(int size)  {                       // 创建顺序栈实例
                mSize = size;
                st = new T[mSize];
                top = -1;
        }
```

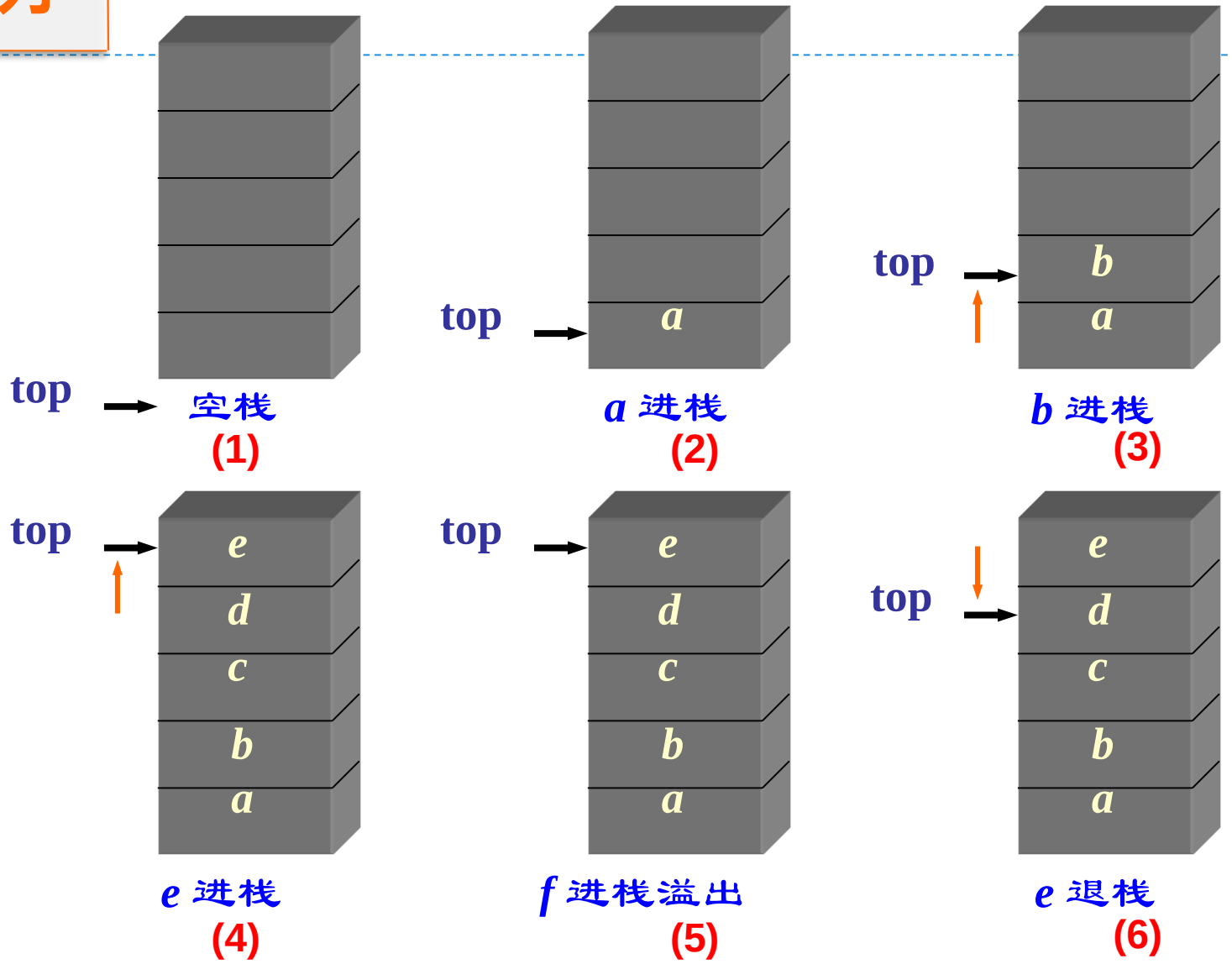
```
    arrStack() {                    // 创建一个顺序栈的实例
        top = -1;
    }
    ~arrStack() {                   // 析构函数
        delete [] st;
    }
    void clear() {                  // 清空栈内容
        top = -1;
    }
}
```

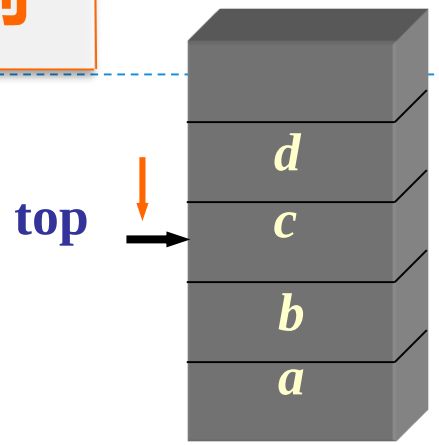
运算集

- 入栈
- 出栈
- 从栈顶读取，但不弹出
- 变空栈
- 栈满时，返回非零值

顺序栈的生成方向

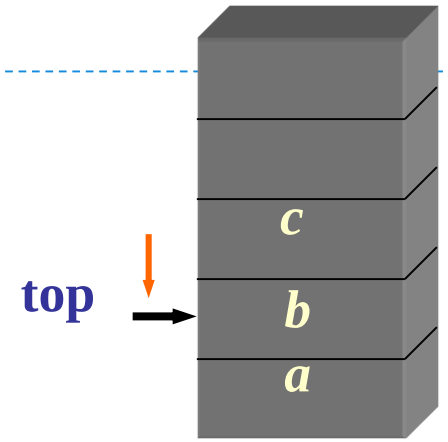
- 若入栈动作使地址向高端生长，称为“向上生成”的栈
- 若入栈动作使地址向低端生长，称为“向下生成”的栈
- 通常我们使用“向上生成”的栈，即：
  - 空栈：top==base （t==-1）
  - 入栈后：top+1 (t++)
  - 出栈后：top-1 (t--)
  - 栈满：top-base==stacksize (t==MAXNUM-1)


top
空栈
(1)
top
a
a 进栈
(2)
top
b
a
b 进栈
(3)
top
e
d
c
b
a
e 进栈
(4)
top
e
d
c
b
a
f 进栈溢出
(5)
top
e
d
c
b
a
e 退栈
(6)



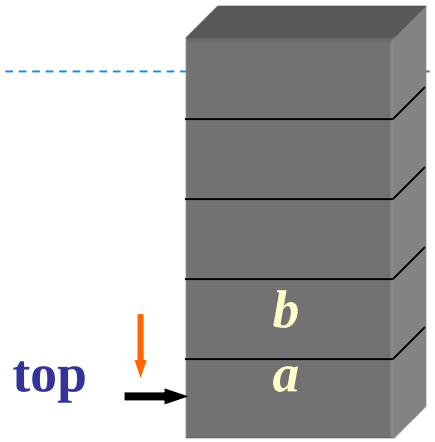
d 退栈

(7)



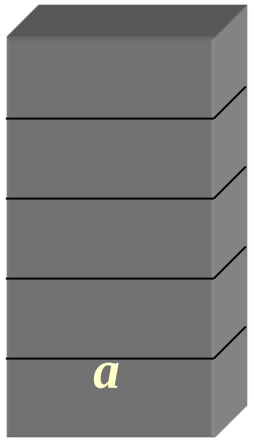
c 退栈

(8)



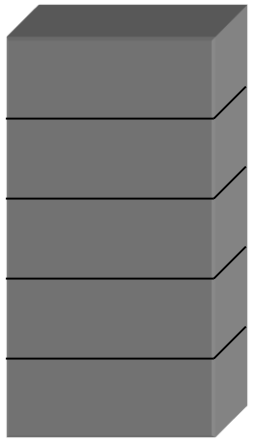
b 退栈

(9)



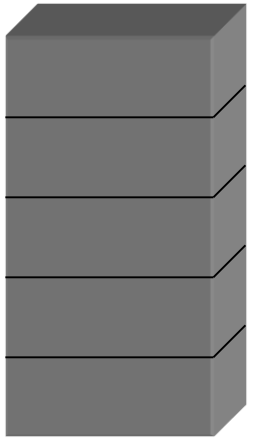
a 退栈

(10)



空栈

(11)



退栈溢出

(12)

顺序栈的溢出

- 上溢（Overflow）
  - 当栈中已经有maxsize个元素时，如果再做进栈操作，所产生的“无空间可用”现象
- 下溢（Underflow）
  - 空栈进行出栈所产生的“无元素可删”现象



入栈

```
bool arrStack<T>::push(const T item) {
   if (top == mSize-1) {          // 栈已满
        cout << "栈满溢出" << endl;
        return false;
   }
   else {                                  // 新元素入栈并修改栈顶指针
        st[++top] = item;                  "先执行++，而后执行进栈操作！"
        return true;
   }
}
```

出栈

```
bool arrStack<T>::pop(T & item) {          // 出栈的顺序实现
   if (top == -1) {                        // 栈为空
        cout << "栈为空，不能执行出栈操作"<< endl;
        return false;
   }
   else {
        item = st[top--];                  // 返回栈顶元素并修改栈顶指针
        return true;
   }
}
```

从栈顶读取，但不弹出

```
bool arrStack<T>:: top(T & item) {
        // 返回栈顶内容，但不弹出
   if (top == -1) {                    // 栈空
        cout << " 栈为空，不能读取栈顶元素"<< endl;
        return false;
   }
   else {
        item = st[top];
        return true;
   }
}
```



其他算法

清空栈

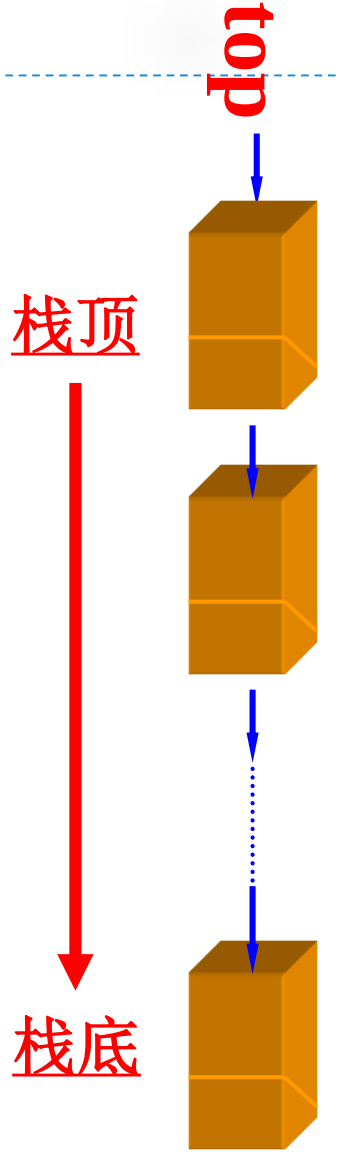
```
void  arrStack<T>::clear() {
    top = -1;
}
```

判断栈满：返回非零值（真值true）

```
bool arrStack<T>:: isFull() {
   return  (top == mSize-1) ;
}
```

3.1.3 链式栈

- 链式栈
  - 栈的链式存储结构，是运算受限的链表
  - 指针方向：从栈顶向栈底
  - 只能在链表头部进行操作，故没有必要像单链表那样附加头结点。
  - 栈顶指针就是链表的头指针（top）
  - 无栈满问题（但存在栈空约束）



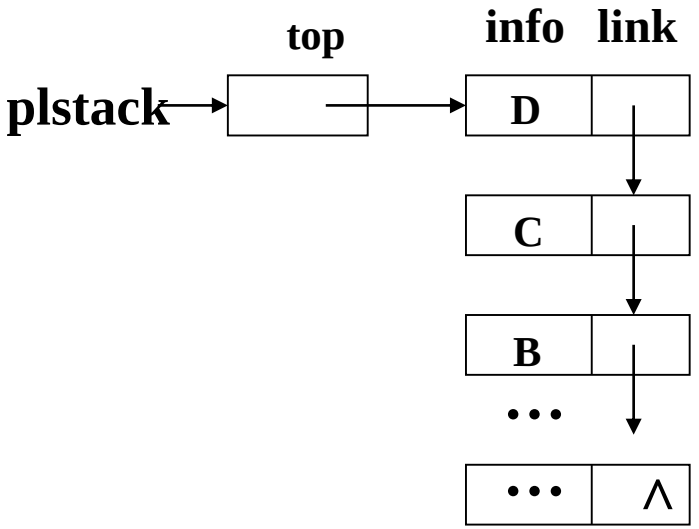
链栈的表示

```
struct  Node;                          /*单链表结点结构*/
typedef struct Node *PNode;    /*指向结点的指针类型 */
struct Node {
  DataType  info;
  PNode   link;                  /* 下一个结点 */
};
```

```
struct LinkStack  {
   PNode  top;
   /* 栈顶指针，指向栈顶结点 */
  };
typedef struct LinkStack *PLinkStack;
   /*链栈类型的指针类型*/
```

- PLinkStack plstack;
- plstack->top: 指向栈顶的指针
- plstack->top->info: 栈顶元素值

- 栈空条件: plstack->top = NULL
- 下溢条件: if (plstack->top == NULL)
  还要继续删除



插入和删除元素

- 压入元素：

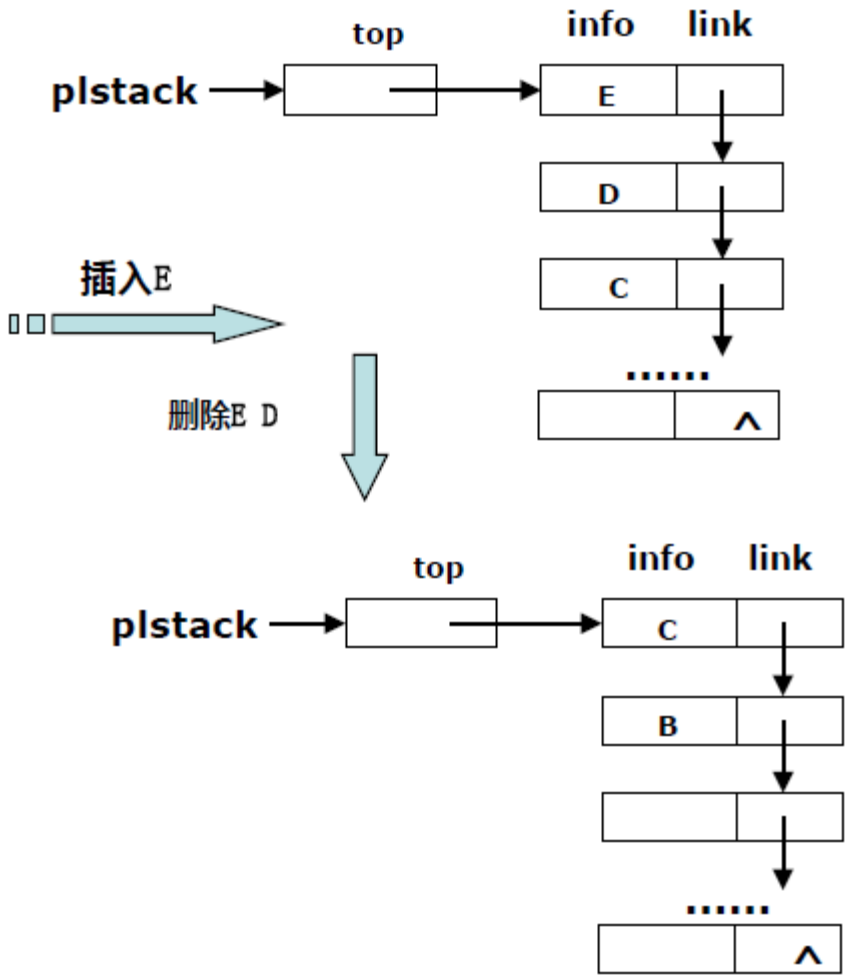
在top与栈顶之间插入

```
(s->link = top
 top = s; )
```

- 弹出元素：

删除栈顶元素

```
( q = top;
  top= q->link;
  free(q);)
```



链式栈的创建

```
template <class T>
class lnkStack : public Stack <T>  {
private:                                          // 栈的链式存储
    Link<T>*        top;                          // 指向栈顶的指针
    int             size;                         // 存放有效元素的个数
public:                                           // 栈运算的链式实现
    lnkStack(int defSize) {                       // 构造函数
        top = NULL;
        size = 0;
    }
    ~lnkStack() {                                 // 析构函数
        clear();
    }
}
```

运算集：压栈

```
// 入栈操作的链式实现
bool lnkStack<T>:: push(const T item) {
    //创建一个新节点，并使其next域赋值top;
   Link<T>* tmp = new Link<T>(item, top);
    top = tmp;
    size++;
    return true;
}
```

```
Link(const T info, Link* nextValue) {
        data = info;
        next = nextValue;
}
```



运算集：出栈

```
bool lnkStack<T>:: pop(T& item) {//  出栈操作的链式实现
   Link <T> *tmp;
       if (size == 0) {
          cout << "栈为空，不能执行出栈操作"<< endl;
          return false;
       }
       item = top->data;
       tmp = top->next;
       delete top;
       top = tmp;
       size--;
       return true;
}
```

栈链的特点

- 运算受限的单链表，插入删除操作限制在表头位置进行
- 由于只能在链表头部进行操作，因此栈链没有必要像单链表那样添加头节点
- 栈顶指针就是链表的头指针，在头指针指向的地方进行插入删除操作
- 不必预先知道栈的最大尺寸，不必担心栈满（除非整个内存空间已满）。

顺序栈和链式栈的比较

- 时间效率
  - 所有操作都只需常数时间
  - 顺序栈和链式栈在时间效率上难分伯仲
- 空间效率
  - 顺序栈须说明一个固定的长度
  - 链式栈的长度可变，但增加结构性开销
- 实际应用中，顺序栈比链式栈用得更广泛些



3.1.4 栈的应用

- 满足后进先出特性，都可以使用栈，应用广泛
- 常用来处理具有递归结构的数据应用
  - 深度优先搜索
  - 子程序 / 函数调用的管理
  - （消除）递归
- 应用举例

数制转换

- 十进制N和其它进制数的转换是计算机实现计算的基本问题，其解决方法很多，其中一个简单算法基于下列原理：

  N = (N div d)*d +N mod d

  （ 其中:div为整除运算,mod为求余运算）

  例如 $(1348)_{10}=(2504)_8$，其运算过程如下

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2



```
① /*输入一个十进制数，将其转换为八进制数*/
② void conversion( )
③ {
④       PSeqStack  pstack;    int temp;
        pstack =CreateEmptyStack ( 100 );
①       scanf (“%d”,n);
②       while(n)    {
③           push(pstack,n%8);
④           n=n/8;
⑤        }
⑥        while(!isEmptyStack_seq(pstack))   {
⑦          temp =top(pstack);
⑧          pop(pstack);
⑨          printf(“%d”, temp);
⑩        }
⑪}
```

括号匹配检验

- 表达式中允许括号嵌套，如何检查这些括号都是匹配的?

  ()()()()((()())())()()

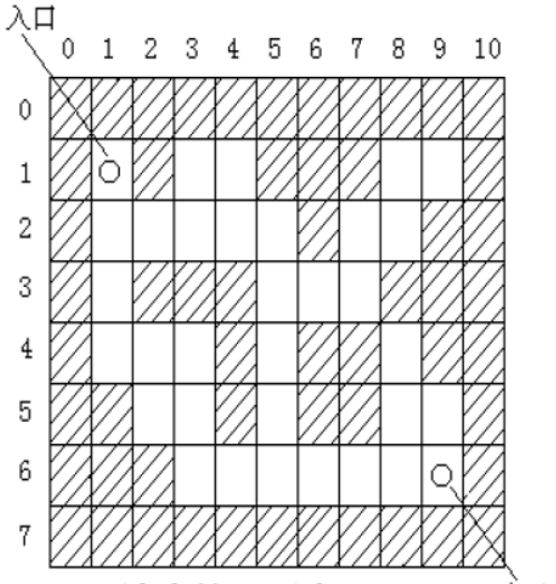
- 如果有其他符号那?

  ()()()([]{}{}{({[]})}) ‘’ “”

```
void CheckMatch(char* p, SeqStack s){
    char c;
    while (*p){
        switch (*p){
        case '(':                    /*如果是左括号，将其入栈*/
            StackPush(&s, *p++);
            break;
        case ')':
            if (StackEmpty(&s)){     /*栈为空，说明没有左括号入栈*/
                printf("缺少左括号！\n");
                return;
            }
            else{
                GetTop(&s, &c);      /*将栈顶元素与读入的右括号比较，如匹配，则将栈顶括号出栈*/
                if (Match(c, *p))
                    StackPop(&s, &c);
                else{   /*栈顶括号与读入的括号不匹配*/
                    printf("左右括号不匹配！\n");
                    return;
                }
            }
        default:   /*如果读入的不是括号，指针后移一位*/
            p++;
        }
    }
    if (StackEmpty(&s))  /*栈为空，所有的字符序列读入完毕，说明括号序列匹配*/
        printf("括号匹配！\n");
    else
        printf("缺少右括号！\n");
}
```

迷宫问题

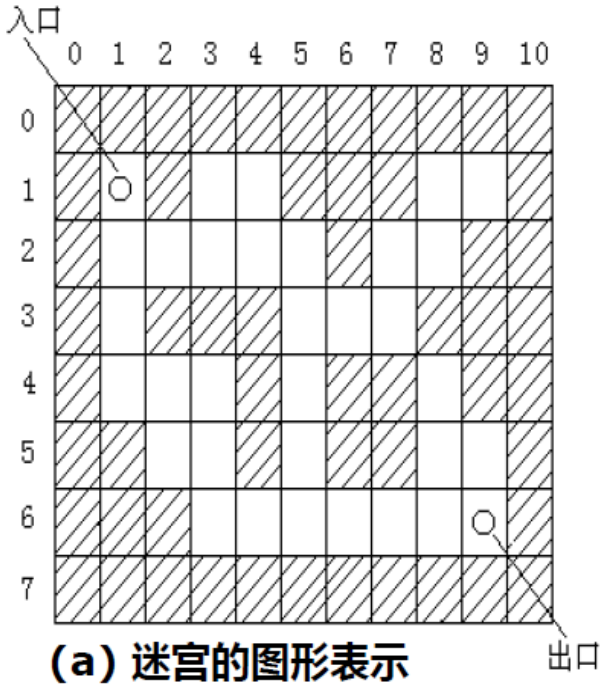
- 在迷宫中求从入口到出口的所有路径是一个经典的程序设计问题。
- 问题分析
  - 迷宫可用下图所示的方块来表示，其中每个元素或为通道（以空白方块表示），或为墙（以带阴影的方块表示）。
  - 只能上下左右运动，要求：从入口到出口的一个以空白方块构成的(无环)路径。



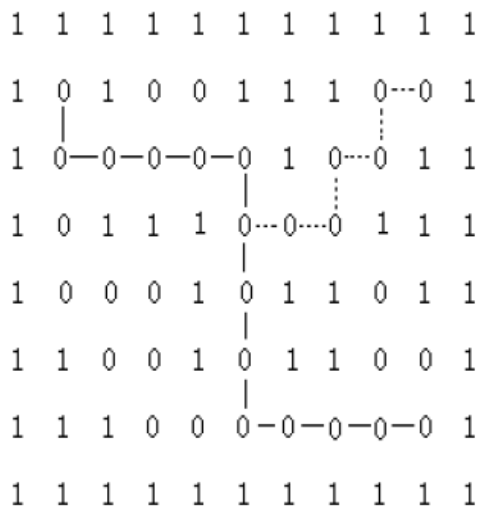
(a) 迷宫的图形表示

程序设计

- 回溯算法：
  - 从入口出发，沿某一方向进行探索，若能走通，则继续向前走；
  - 否则，沿原路返回，换一方向再进行探索，直到所有可能的通路都探索到为止。



(a) 迷宫的图形表示



(b) 迷宫的二维数组表示

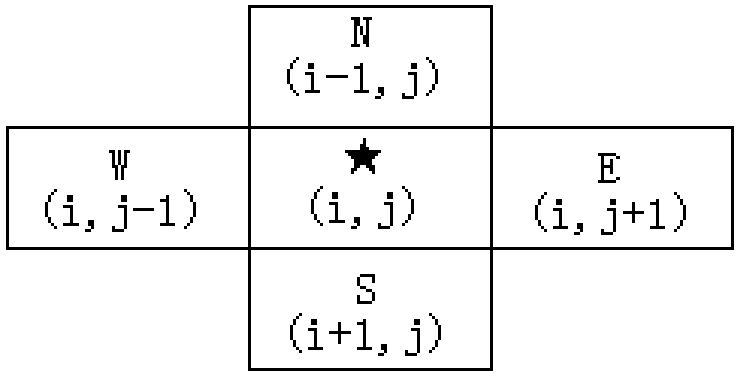
探索-返回对应了入栈-出栈，我们可以利用栈来解决

算法设计

- 数据表示
  - 用二维数组maze[m][n]来表示迷宫，数组中元素为0的表示通道，为1的表示墙。
  - 图中迷宫的入口处为maze[1][1]，出口处为maze[m-2][n-2]；（不同的问题可以自己设定）入口和出口的元素值必为0；
- 回溯算法：
  - 从入口出发，沿某一方向进行探索，若能走通，则继续向前走；
  - 否则，沿原路返回上一位置，换一方向再进行探索，直到所有可能的通路都探索到为止。

算法设计

- 任意时刻在迷宫中的位置maze[i][j]时，可能的运动方向有四个：
  - 要进入E方向的位置(g,h)，要根据由该增量值表来修改坐标，即 g = i + direction[0][0]; h = j + direction[0][1];
  - 若(i,j)为(3,4)，则其E方向的相邻点为(3,5)。



direction[4][2]

0	0	1	方向 E
1	1	0	方向 S
2	0	-1	方向 W
3	-1	0	方向 N

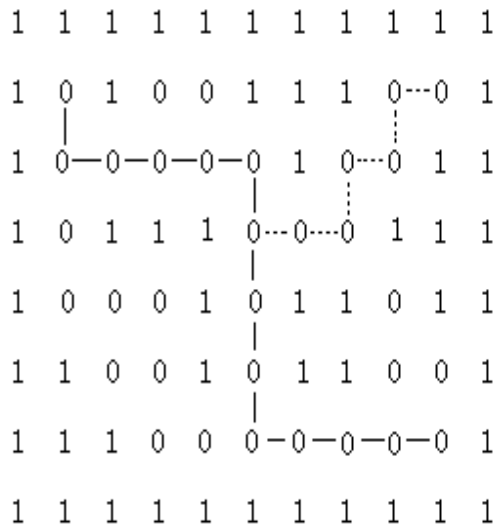
算法设计

- 经过上述设计，求迷宫中一条路径的算法，可以
  - 从入口开始，对每个“当前位置”都从E方向试起，若不能通过，则顺时针试S方向、W方向、N方向。
  - 为避免走回到已进入的点（包括已在当前路径上的点和曾经在当前路径上的点），凡是进入过的点都应做上记号，避免产生死循环。
  - 为记录当前位置及在该位置上所选的方向，设置一个栈，栈中每个元素包括三项，分别记录当前位置的行坐标、列坐标及在该位置上所选的方向。

```
typedef struct NodeMaze { int x, y, d; }  DataType;
```

回溯算法框架

```
mazeFrame( void )  //其算法框架如下= =
{
   创建一个（保存探索过程的）空栈;
   把入口位置入栈;
   while  栈不空时
    {
       取栈顶位置并设置为当前位置;
       while 当前位置存在试探可能
       {
           取下一个试探位置;
           if (下一个位置是出口)  打印栈中保存的探索过程然后返回;
           if(下一个位置是通道)
                    把下一个位置进栈并且设置为的当前位置;
        }
        弹出栈顶元素     // 实现回溯
     }
}
```



栈与递归

- 递归概念
  - 是指在调用一个函数的过程中又直接调用或间接调用了函数自身

➢直接调用

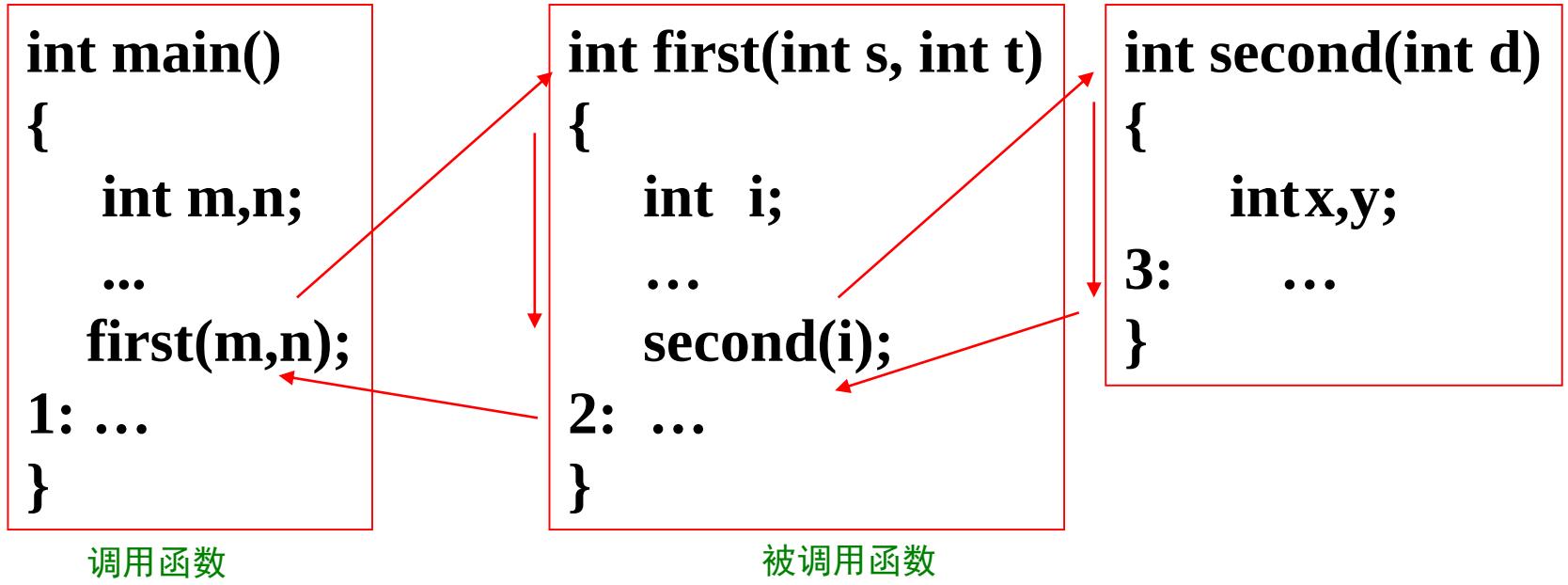
```
long fact(int n) {//求阶乘
    if (n==1)
        return 1;
    return n*fact(n-1);
}
```

➢间接调用

```
int f1(int a)
   {
      int b;
      b=fn2(a+1);
   }
int f2(int s)
   {
      int c;
      c=fn1(s-1);
   }
```

函数调用的过程

多个函数嵌套调用时，按照“后调用先返回”的原则进行，如下所示：



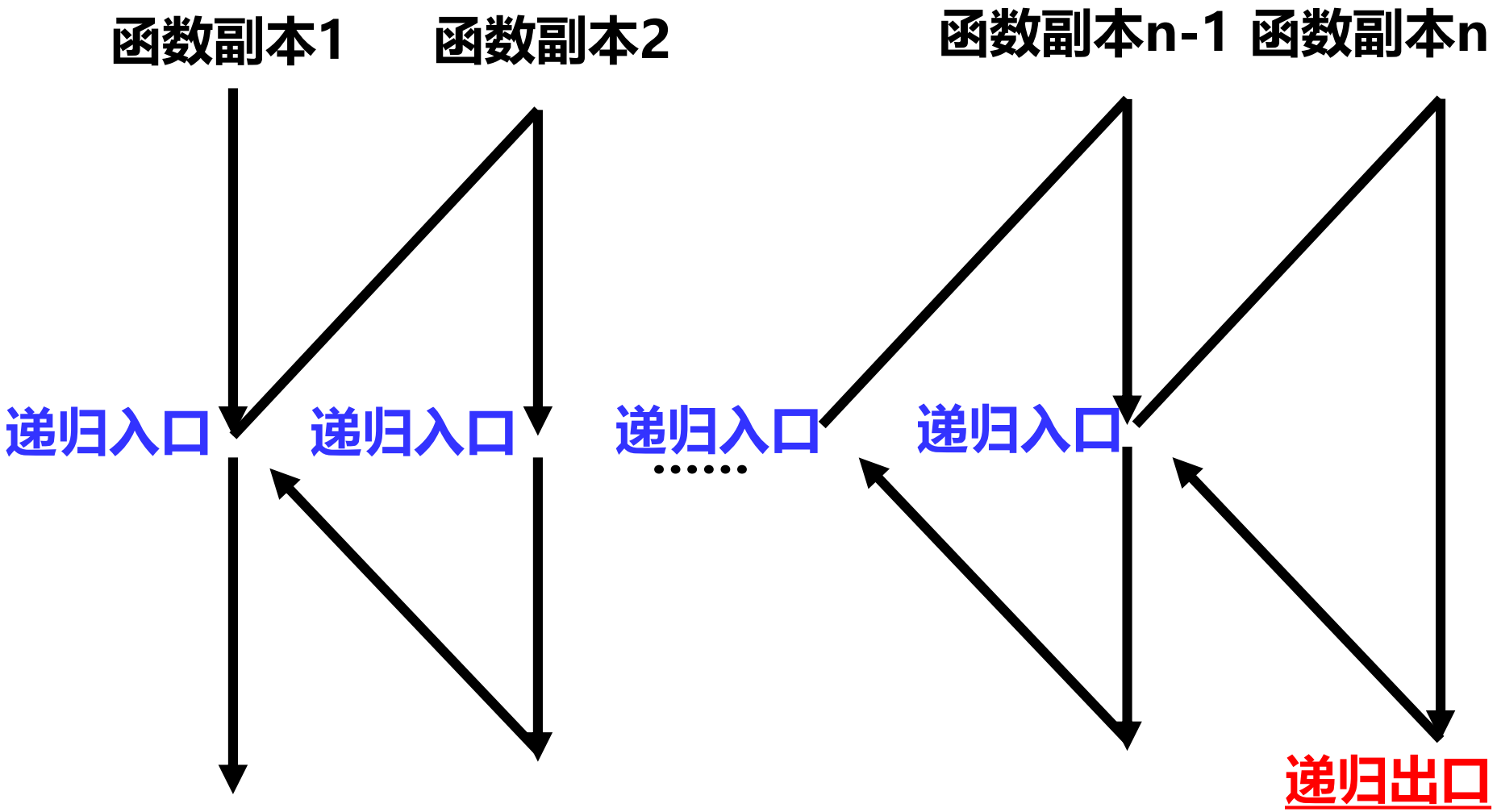
先运行后出结果，后运行先出结果，类似栈的思想，因此和栈有关系

函数调用的过程

- 调用前：
  ① 调用函数将实参、返回地址传递给被调用函数
  ② 为被调用函数分配必要的数据区（存放局部变量、实参、返回地址等），接收调用函数传送来的调用信息
  ③ 将控制转移到被调用函数入口。

- 调用后：
  ① 传送返回信息，如被调用函数的计算结果。
  ② 释放被调用函数的数据区
  ③ 把控制转移到调用函数中

递归调用过程示例



递归的实现

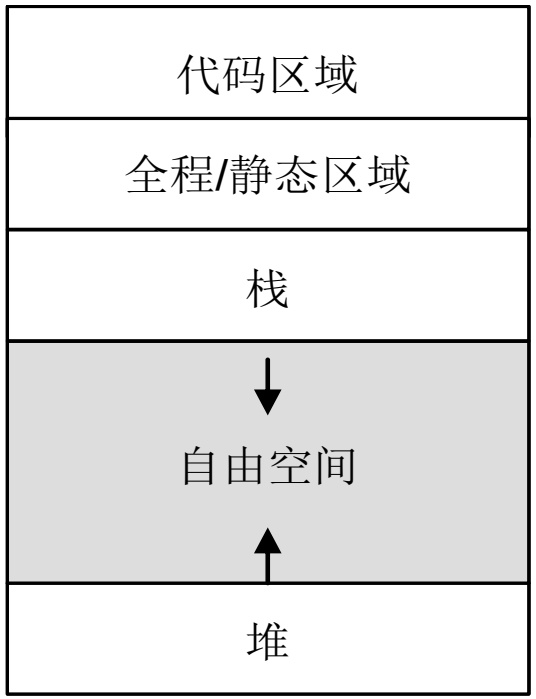
- 一个问题能否用递归实现，看其是否具有下面特点
  - 有递推公式（1个或多个）
  - 有递归结束条件（1个）
- 编写递归函数时，程序中必须有相应的语句
  - 一个（或者多个）递归调用语句
  - 测试结束语句
  - 先测试，后递归调用
- 递归程序的特点
  - 易读、易编，但占用额外内存空间。

函数运行时的存储分配

- 静态分配
  - 在非递归情况下，数据区的分配可以在程序运行前进行，一直到整个程序运行结束才释放，这种分配称为静态分配。
  - 采用静态分配时，函数的调用和返回处理比较简单，不需要每次分配和释放被调函数的数据区
- 动态分配
  - 在递归（函数）调用的情况下，被调函数的局部变量不能静态地分配某些固定单元，而必须每调用一次就分配一份，以存放当前所使用的数据，当返回时随即释放。【大小不确定，值不确定】
  - 动态分配在内存中开辟一个称为运行栈的足够大的动态区

动态存储分配

- 用作动态数据分配的存储区，分为堆（heap）和栈（stack）
  - 堆区域则用于不符合LIFO（诸如指针的分配）的动态分配
  - 栈用于分配发生在后进先出LIFO风格中的数据（诸如函数的调用）



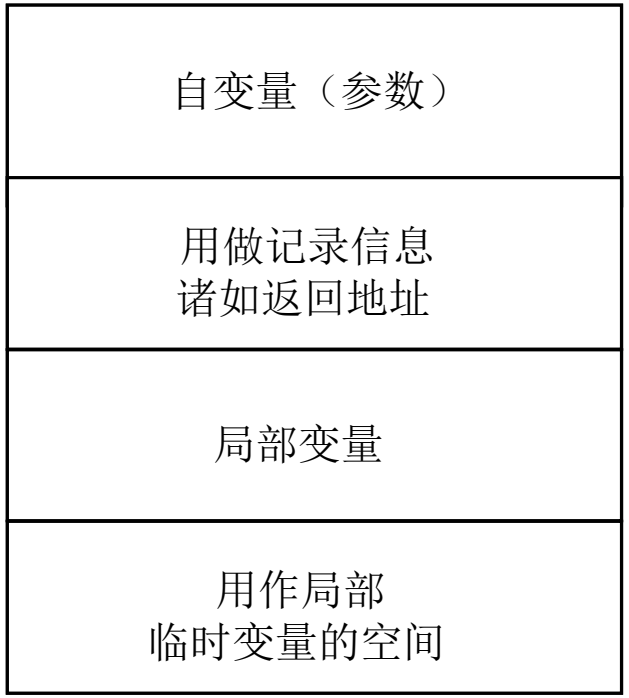
存放全局变量和静态变量

存放函数的形式参数/局部变量，由编译器分配并自动释放

用malloc或者new申请的内存空间，需要手工释放

运行栈中的活动记录

- 函数活动记录是动态存储分配中的基本单元
  - 当调用函数时，函数的活动记录包含为其局部数据分配的存储空间



运行栈中的活动记录

- 运行栈随着程序执行时发生的调用链或生长或缩小
  - 每次调用执行进栈操作，把被调函数的活动信息压入栈顶
  - 函数返回执行出栈操作，恢复到上次调用所分配的数据区
- 一个函数在运行栈上可以有若干不同的活动记录，每个都代表了一个不同的调用
  - 递归深度决定运行栈中活动记录的数目
  - 同一局部变量在不同的递归层次被分配给不同的存储空间
  - 对应代码运行多次，但是没有被复制多次！

举例1：阶乘n！

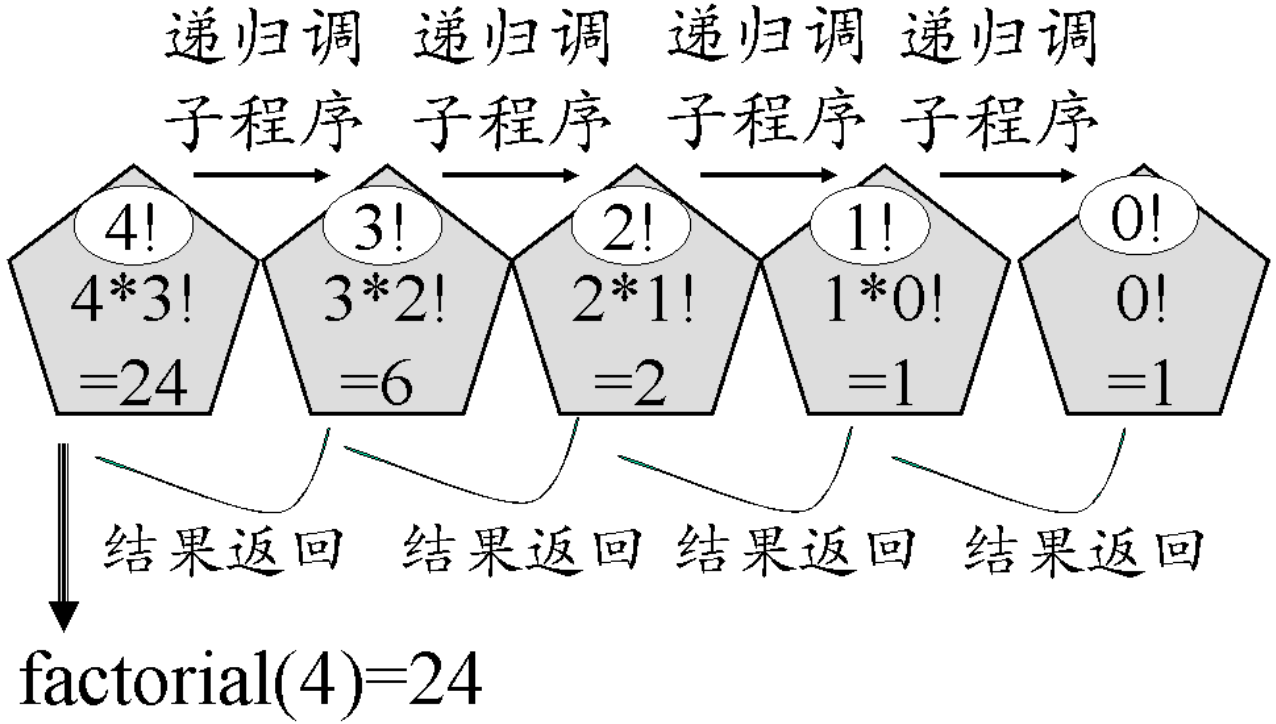
- 阶乘n！的递归定义

$$factorial(n+1)=\begin{cases}1, & if\ n\le 0\\ \text{n*factorial(n-1)}, & \text{if n}>0\end{cases}$$

- 递归入口
  - factorial(n-1)
- 递归出口
  - n≤0

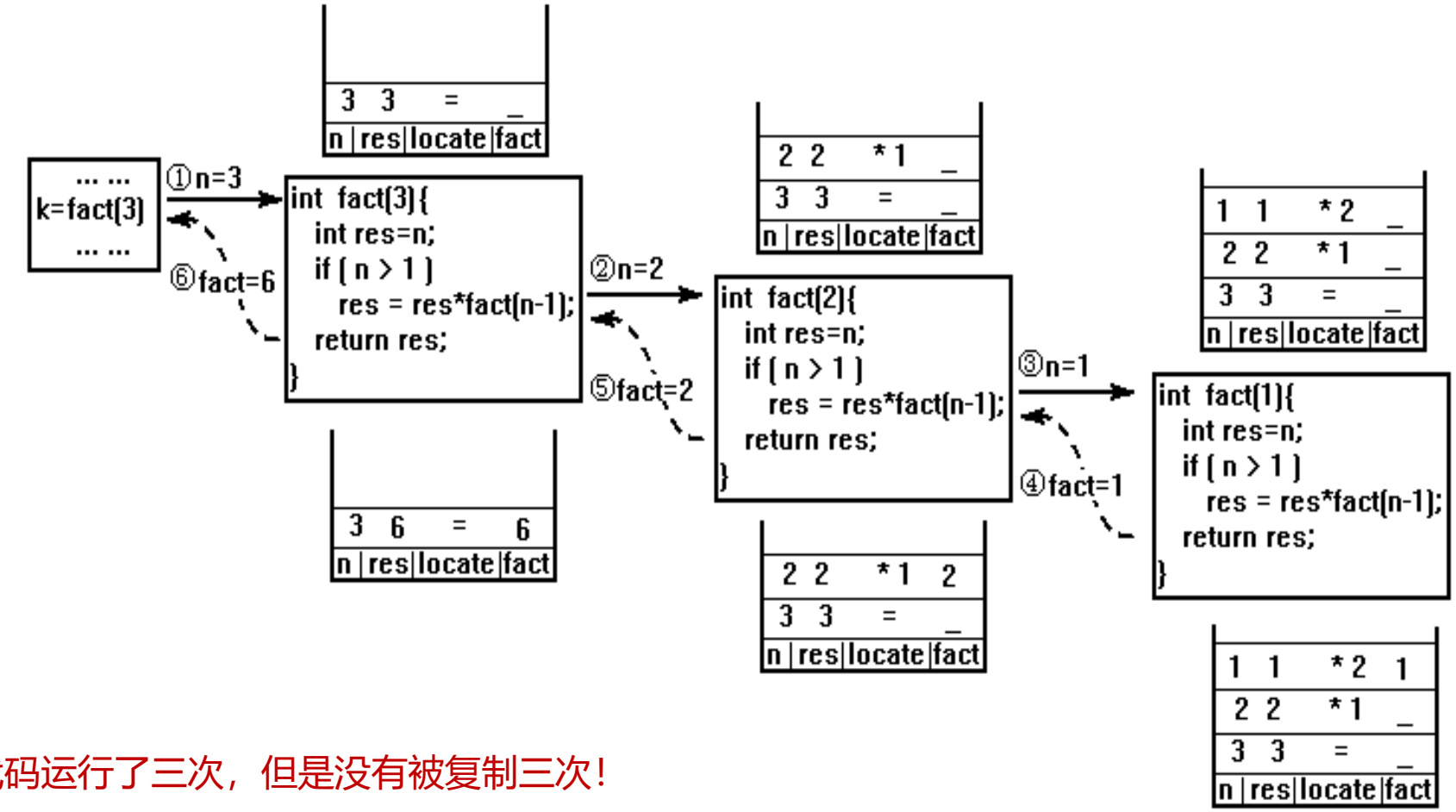
程序实现

```
//递归函数
long factorial(long n)
{
    if (n==0)
        return 1;
    else
     //递归调用
    return n * factorial( n-1) ;
}
```



递归过程示意图(4!)

- 当n为0时定义为1，它不再用递归来定义，称为递归定义的出口，简称为递归出口。



fact()对应代码运行了三次，但是没有被复制三次！

阶乘的非递归计算

```
①  int  nfact( int n )
②  {
③          int res;
④          PSeqStack  st;                  /* 使用顺序存储结构实现的栈 */
⑤          st = createEmptyStack( );

⑥          while (n>0)                     /* 按照调用次序，压栈 */
⑦          {          push_seq(st,n);  n = n – 1;          }
⑧
⑨          res = 1;                        /* 按照调用的反次序，退栈 */
⑩          while (! isEmptyStack_seq (st))
⑪          {          res = res * top_seq(st);  pop_seq(st);        }
⑫
⑬          free(st);
⑭          return ( res );
⑮  }
```

表达式的求值

- 表达式的组成
  - 基本符号集：由{0，1，…，9，+，-，*，/，（，）}等16个基本符号组成；
  - 语法成分集：由{ <表达式>，<项>，<因子>，<常数>，<数字> }5个语法成分组成；
  - 语法公式集 ：“::=”是规则定义符。
- 一个表达式由操作数(亦称运算对象)、操作符（亦称运算符）和分界符组成。

表达式的分类

- 算术表达式有三种表示：
  - 中缀(infix)表示

    <操作数> <操作符> <操作数>，如 A+B；
  - 前缀(prefix)表示

    <操作符> <操作数> <操作数>，如 +AB；
  - 后缀(postfix)表示

    <操作数> <操作数> <操作符>，如 AB+；

31 * (5 - 22) + 70　　+*31-5 22 70　　31 5 22 - * 70 +



中缀表达式的递归定义

- 中缀计算表达式的递归定义
  - <表达式> :: = <项> + <项> | <项> - <项> | <项>
  - <项> :: = <因子> * <因子> |<因子> / <因子> |<因子>
  - <因子> :: = <常数> | (<表达式>)
  - <常数> :: = <数字> |<数字> <常数>
  - <数字> :: = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9



中缀表达式的计算次序

- (1)先执行括号内的计算，后执行括号外的计算
  - 在具有多层括号时，按层次反复地脱括号
  - 左右括号必须配对
- (2)在无括号或同层括号时，先乘(*) 、除(/)，后作加(+)、减(-)
- (3)在同一个层次，若有多个乘除（*、 / ）或加减 （+， -)的运算，那就按自左至右顺序执行

后缀表达式（逆波兰式）

- <表达式>::= <项><项>+|<项><项> - |<项>
- <项>::=<因子><因子>*|<因子><因子>/|<因子>
- <因子> ::= <常数>
- <常数>::= <数字>|<数字><常数>|
- <数字>::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

在高级语言程序设计中，使用后缀表达式进行算术表达式的求值计算！

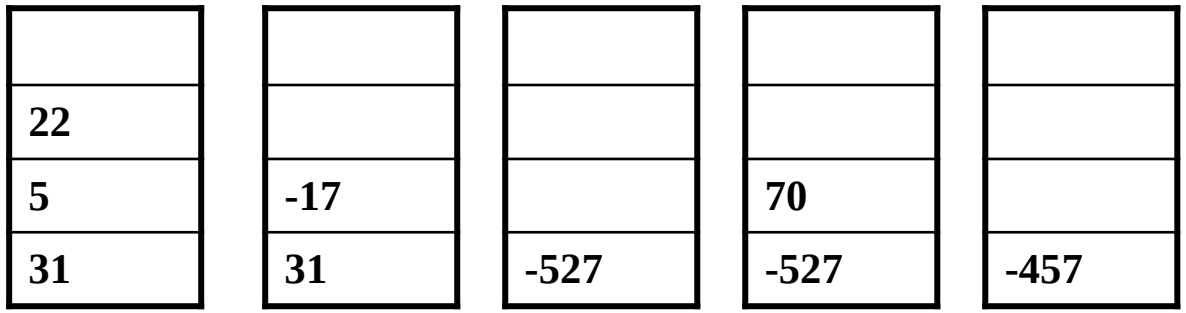
后缀表达的计算规则

- 后缀表达式不含括号！
- 运算符放在两个参与运算的语法成分的后面；
- 所有求值计算皆按运算符出现的顺序，严格从左向右进行。

后缀表达式的求值

- 后缀表达式的主要优点是可以写出非常简单的求值过程。
- 需要一个存放操作数的栈：
  - 从左往右扫描表达式，
  - 遇到操作数进栈；
  - 遇到运算符时从栈中弹出两个操作数计算，并将计算的结果再压入栈。
  - 扫描结束时，栈顶元素就是最后的结果。

31 5 22 - * 70 +



如何转换?

- 在中缀和后缀表达式中
  - 操作数的次序保持不变、操作符的次序发生改变
- 中缀表达式中，操作符的次序受下列因素影响
  - 先括号内、后括号外
  - 先乘除而后加减
  - 同级别先左后右
- 后缀表达式中，操作符的次序要与中缀表达式中的计算次序保持一致！！

如何保持一致？ 借助栈进行排序

中缀到前后缀表达式的转换

- 转化步骤：
  - 按照运算符的优先级对所有的运算单位加括号
  - 将运算符移动到对应括号的前面（前缀表达式）或后面（后缀表达式）
  - 去掉括号，得到前缀或后缀表达式

□ 示例：31 * (5 - 22) + 70

- 加括号 ((31*(5-22))+70)
- 移动运算符

  对于前缀表达式，变成 +(*(31-(5 22))70)

  对于后缀表达式：变成((31(5 22)-)*70)+
- 去掉括号

  前缀表达式： +*31-5 22 70

  后缀表达式：31 5 22 - * 70 +

中缀到后缀表达式的转换

思路：利用栈来记录表达式中的运算符

- 当输入是操作数，直接输出到后缀表达式序列
- 当输入的是左括号时，也把它压栈
- 当输入的是运算符时
  - While（以下循环）
    - If（栈非空 *and* 栈顶不是左括号 *and* 输入运算符的优先级“≤”栈顶运算符的优先级）时
      - 将当前栈顶元素弹栈，放到后缀表达式序列中（此步反复循环，直到上述if条件不成立）；将输入的运算符压入栈中。
    - Else　把输入的运算符压栈（>当前栈顶运算符才压栈！）

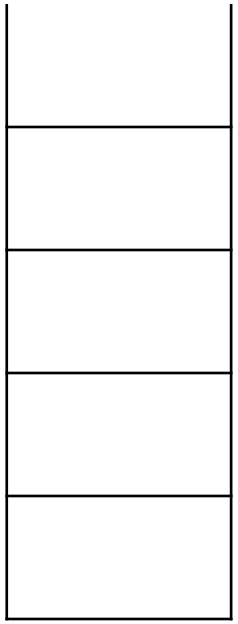
…转换(续)

- 当输入的是右括号时，先判断栈是否为空
  - 若为空（括号不匹配），异常处理，清栈退出；
  - 若非空，则把栈中的元素依次弹出
    - 遇到第一个左括号为止，将弹出的元素输出到后缀表达式的序列中（弹出的开括号不放到序列中）
    - 若没有遇到左括号，说明括号也不匹配，做异常处理，清栈退出
- 最后，当中缀表达式的符号序列全部读入时，若栈内仍有元素，把它们全部依次弹出，都放到后缀表达式序列尾部。
  - 若弹出的元素遇到左括号时，则说明括号不匹配，做错误异常处理，清栈退出

中缀表达式→后缀表达式

输入中缀表达式： 23 ＋ (34 * 45) / (5 ＋ 6 ＋ 7)

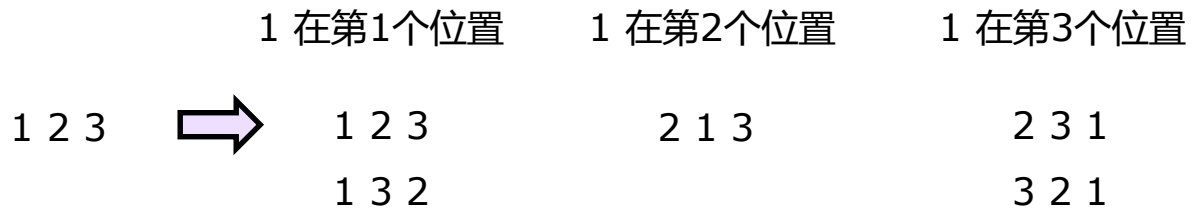
栈的状态



输出后缀表达式：

给定一个入栈序列，序列长度为N，请计算由多少种出栈序列?

- 关键是写出F(N)的递推公式
  - F(0)=1, F(1) = 1, F(2) = 2
  - F(3) = ?



- 对于长度为N序列的情况

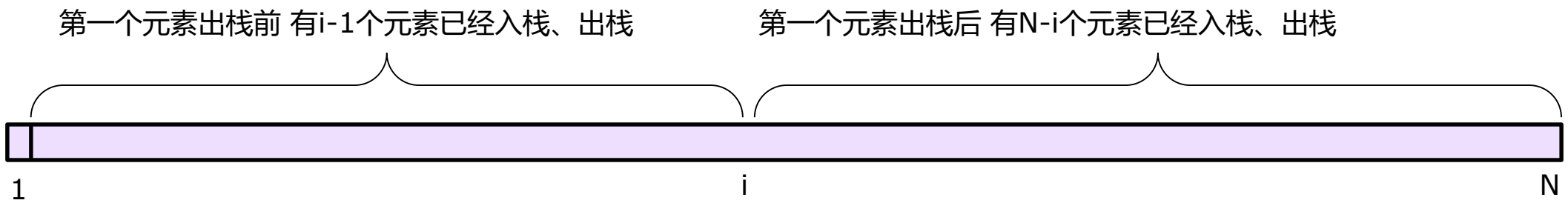
  如果第一个元素在第i个位置出栈 $F_i(N) = A_i(N)XB_i(N)$

  总的出栈序列个数: $F(N) = \sum_{i=1}^{N} F_i(N)$

如何计算 $A_i(N)$和$B_i(N)$?

给定一个入栈序列，序列长度为N，请计算由多少种出栈序列?

如果第一个元素在第i个位置出栈 $F_i(N) = A_i(N) X B_i(N)$



第1个元素对其后面i-1个元素的入、出栈次序没有约束

$A_i(N) = F(i-1)$

第1个元素对其后面N-1个元素的入、出栈次序没有约束

$B_i(N) = F(N-i)$

$$F(N) = \sum_{i=1}^{N} F(i-1) * F(N-i)$$

$$C_0 = 1 \quad \text{and} \quad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i} \quad \text{for } n \geq 0. \qquad C_n = \frac{1}{n+1}\binom{2n}{n}$$

计算器类的实现

```
// Class Declaration 类的说明
class Calculator  {
private:
      Stack<double> s;     // 这个栈用于压入保存操作数
      bool GetTwoOperands(double& opd1,double& opd2); // 弹出两个操作数
      void Compute(char op); // 取两个操作数，并按op对两个操作数进行计算
public:
      Calculator(void){} ;// 创建计算器实例，开辟一个空栈
      void Run(void);      // 读入后缀表达式，遇到符号"="时，求值计算结束
      void Clear(void);    // 计算器的清除，为随后的下一次计算做准备
};
```

部分成员函数的程序实现

```
bool Calculator::GetTwoOperands(double& opd1, double& opd2)  {
       if (s.isEmpty()) {
              cerr << "Missing operand!" <<endl;
              return false;
       }
        s.pop(&opd1);                                  // 右操作数
       if (s.isEmpty())  {
              cerr << "Missing operand!" <<endl;
              return false;
       }
       s.pop(&opd2) ;                          // 左操作数
       return true;
}
```

```
void Calculator::Compute (char op)
{
   Boolean result;
   double operand1, operand2;
   result = GetTwoOperands(operand1,operand2);
   if(result == True)
         switch(op)
         {
               case '+': S.Push(operand2 + operand1);
                         break;
               case '-': S.Push(operand2 - operand1);
                         break;
               case '*': S.Push(operand2 * operand1);
                         break;
               case '/':
                         if(operand1 == 0.0)
                         {
                               cerr<<"Divided by 0!"<<endl;
                               S.ClearStack();
                         }
                         else
                               S.Push(operand2 / operand1);
               break;
          }
        else
               S.ClearStack();
}
```

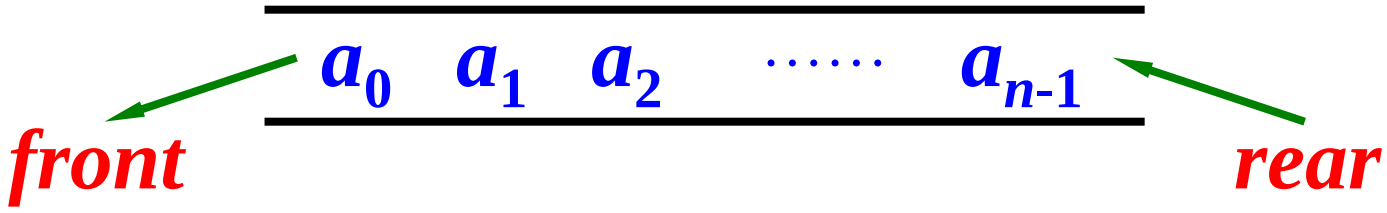
```
void Calculator::Run(void)
//读入表达式，遇到操作数则入栈，遇到操作符则从栈
    中连续弹出两个操作数计算，直至遇到“=”结束。
{
    char c;
    double newoperand;
    while(cin>>c, c != '=')  {
          case '+':
          case '-':
          case '*':
          case '/':
                  Compute(c);
                  break;
          default:
                  cin.pushback(c);
                  cin >> newoperand;
                  S.push(newoperand);
                  break;
    }
          if (!S.IsEmpty())
             cout << S.top()  << endl; //印出求值的最后结果
}  //end calculator::run(void)

void Calculator::Clear(void)
{
    S.ClearStack();
}
```

[后缀计算器的类定义，结束]

3.2 队列(Queue)

- 队列是只允许在一端删除，在另一端插入的线性表
- 允许删除的一端叫做队头(front)，允许插入的一端叫做队尾(rear)
- 特性
  - 先进先出(FIFO, First In First Out)





队列的主要操作

- 入队列（enQueue）
- 出队列（deQueue）
- 取队首元素（getFront）
- 判断队列是否为空（isEmpty）

抽象数据类型

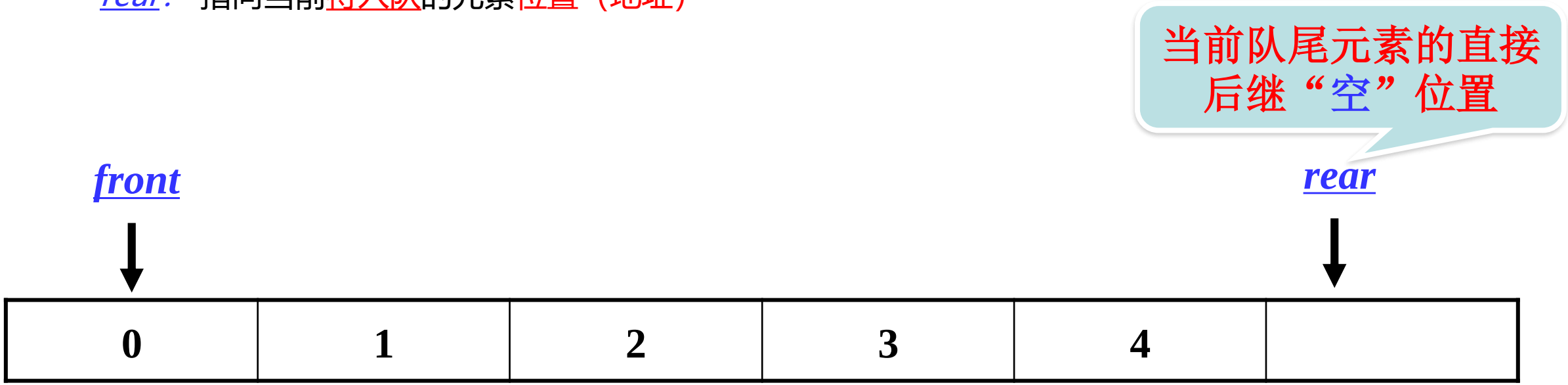
```
template <class T>
class Queue  {
public:                                  // 队列的运算集
   void clear();                         // 变为空队列
   bool enQueue(const T item);           // 将item插入队尾，成功则返回真，否则返回假
   bool deQueue(T& item);                // 返回队头元素并将其从队列中删除，成功则返回真
   bool getFront(T& item);               // 返回队头元素，但不删除，成功则返回真
   bool isEmpty();                       // 返回真，若队列已空
   bool isFull();                        // 返回真，若队列已满
};
```

队列的实现方式

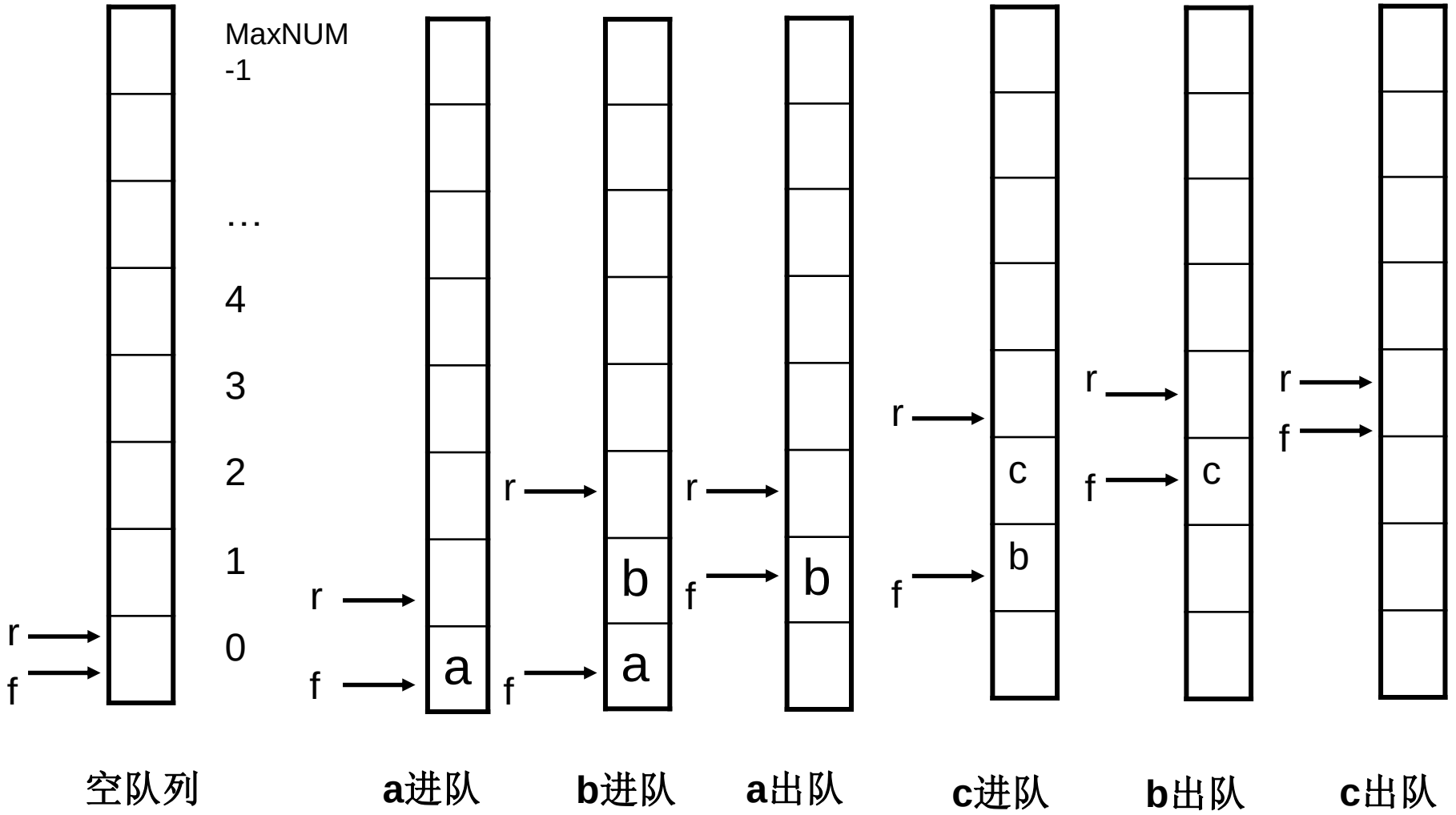
- 顺序队列
  - 顺序表实现
- 链式队列
  - 单链表实现

3.2.1 顺序队列

- 用向量存储队列元素，用两个变量分别指向队列的前端(front)和尾端(rear)
  - *front*：指向当前待出队的元素位置（地址）
  - *rear*：指向当前待入队的元素位置（地址）



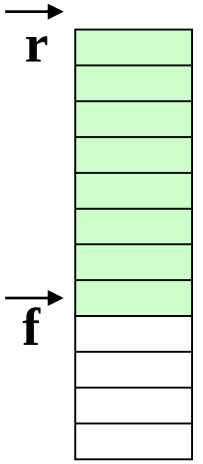
顺序队列示意



普通顺序队列的缺陷

- 队列溢出：
  - 当队列满时，再做进队列操作，这种现象称为上溢
  - 当队列空时，再做删除操作，这种现象称为下溢
- 插入
  - `elem[r] = x`
  - `r = r+1`
- 删除
  - `f = f+1`
- 当 `r ==MAXNUM`
  - 再做插入就会产生溢出，而实际上这时队列的前端还有许多空的可用的位置，这种现象称为假溢出。

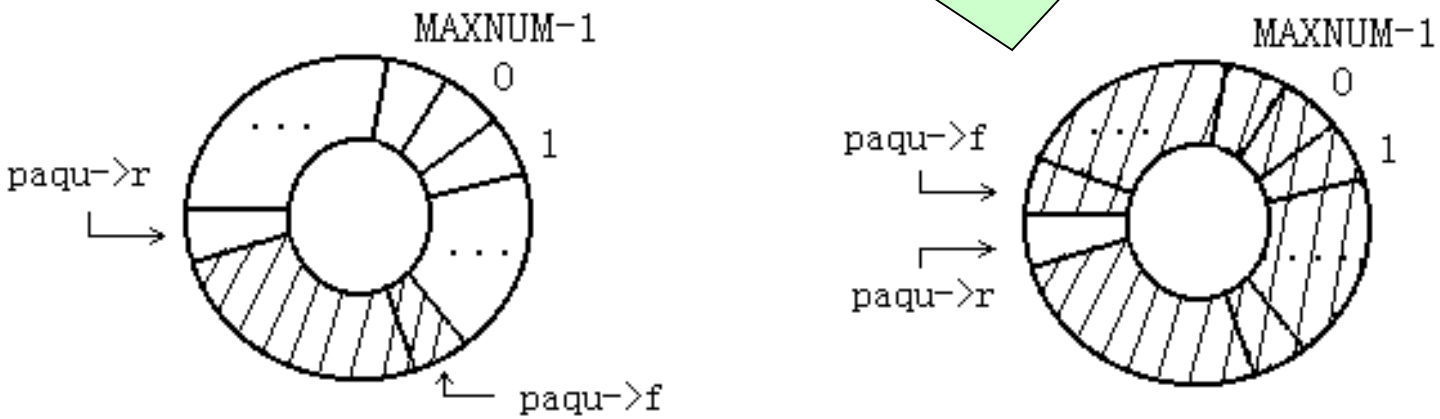
先插入，后增加尾变量

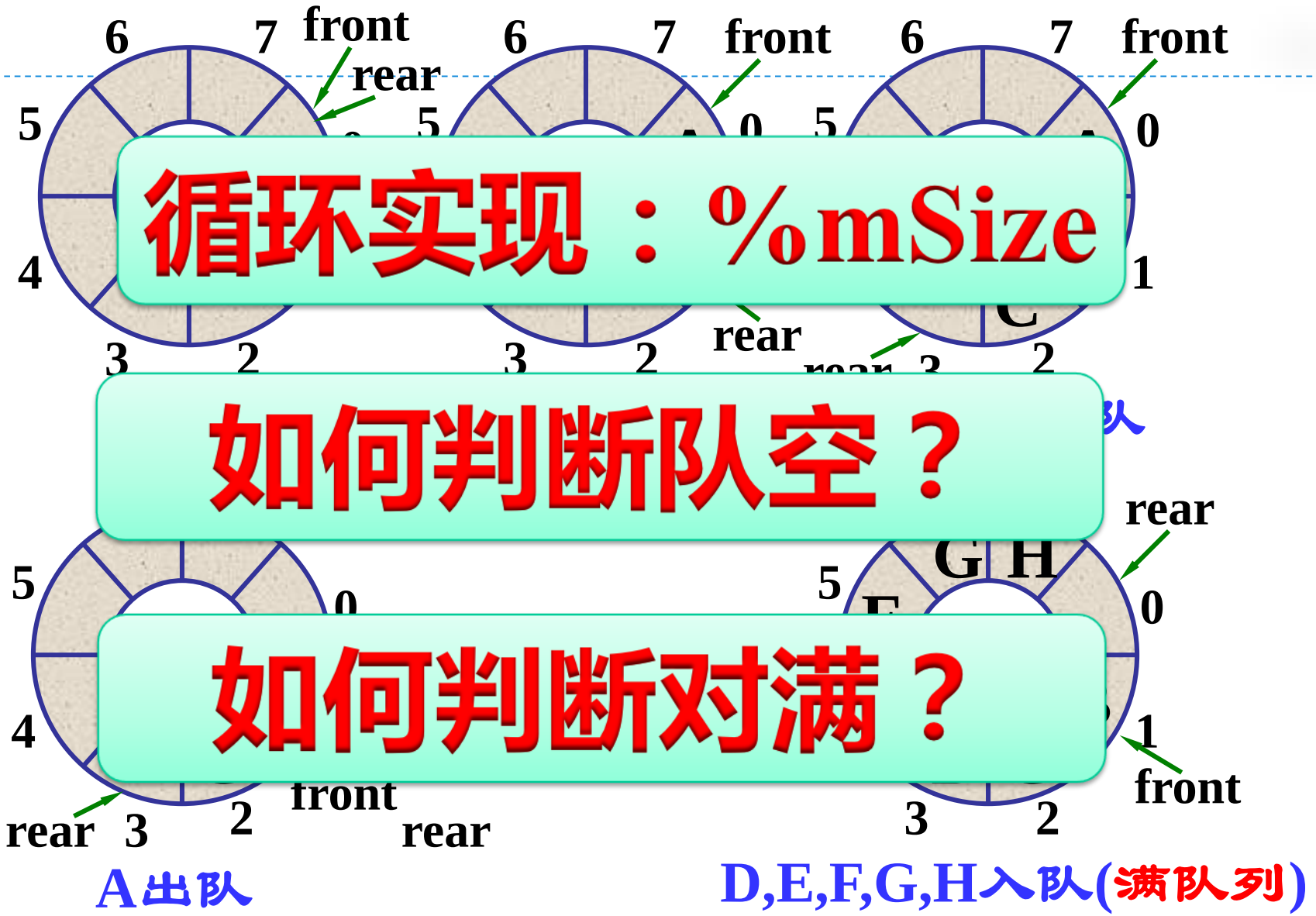


循环队列

- 解决方法：
  - 把数组elem[MAXNUM]从逻辑上看成一个环
- 循环队列判空满条件：
  - 附设一变量，头尾相碰时判断头追上尾还是尾追上头
  - 少用一个空间，当尾+1等于头时为满；当头等于尾时为空

队空条件：f = = r;
队满条件：(r+1)%MAXNUM == f




循环实现：%mSize
如何判断队空？
如何判断对满？
front
rear
A出队
D,E,F,G,H入队(满队列)

队列的类定义

```
class arrQueue: public Queue<T> {
    private:
      int     mSize;                    // 存放队列的数组的大小
      int     front;                    // 表示队头所在位置的下标
      int     rear;                     // 表示待入队元素所在位置的下标
      T       *qu;                      // 存放类型为T的队列元素的数组
    public:                             // 队列的运算集
       arrQueue(int size)  {            // 创建队列的实例
          mSize = size +1;              // 浪费一个存储空间，以区别队列空和队列满
          qu = new T [mSize];
          front = rear = 0;
       }
       ~arrQueue()  {                   // 消除该实例，并释放其空间
       delete [] qu;
       }
}
```

入队列操作

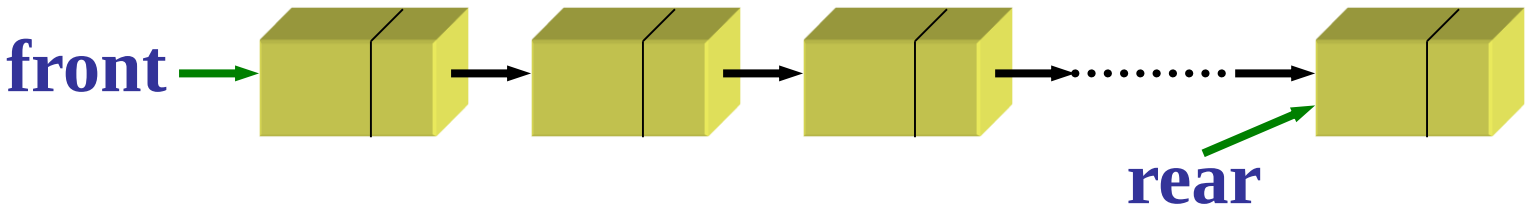
```
bool arrQueue<T> :: enQueue(const T item)  {
                                //  item入队，插入队尾
    if (((rear + 1 ) % mSize) == front) {
            cout << "队列已满，溢出" << endl;
            return false;
    }
    qu[rear] = item;
    rear = (rear +1) % mSize;                //循环后继
    return true;
}
```

出队列操作

```
bool arrQueue<T> :: deQueue(T& item)  {
                    // 返回队头元素并从队列中删除
    if ( front == rear)  {
        cout << "队列为空" << endl;
        return false;
    }
    item = qu[front];
    front = (front +1) % mSize;
    return true;
}
```

3.2.2 链式队列

- 单链表队列：链接指针的方向是从队头指向队尾
- 队头在链头，队尾在链尾
- 链式队列在进队时无队满问题，但有队空问题
- 队空条件： `front == rear==NULL`





队列的类定义

```
template <class T>
class lnkQueue: public Queue <T> {
private:
   int  size;                                  // 队列中当前元素的个数
   Link<T>* front;                             // 表示队头的指针
   Link<T>* rear;                              // 表示队尾的指针
public:                                        // 队列的运算集
   lnkQueue(int size)  {                       // 创建队列的实例
        size = 0;
        front = rear = NULL;
   }
   ~lnkQueue()  {                              // 消除该实例，并释放其空间
        clear();
   }
}
```

运算集：入队

```
bool lnkQueue <T> :: enQueue(const T item) {          //  入队插入队尾
        if (rear == NULL)     {                       // 空队列
                front = rear = new Link<T> (item, NULL);
        }
        else  {                                       // 添加新的元素
                rear-> next = new Link<T> (item, NULL);
                rear = rear ->next;
        }
        size++;
        return true;
}
```

```
Link(const T info, Link* nextValue) {
        data = info;
        next = nextValue;
}
```



运算集：出队

```
// 返回队头元素并从队列中删除
bool lnkQueue <T> :: deQueue(T& item)  {
        Link<T> *tmp;
        if (size == 0)  {              // 队列为空，没有元素可出队
                cout << "队列为空" << endl;
                return false;
        }
        &item = front->data;
        tmp = front;
        front = front -> next;
        delete tmp;
        if (front == NULL)
                rear = NULL;
       size--;
        return true;
}
```

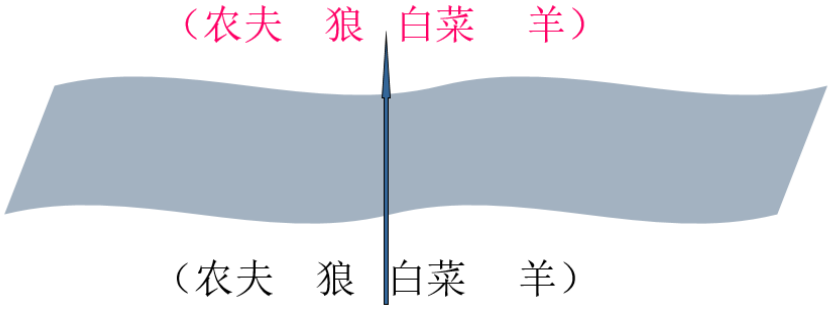


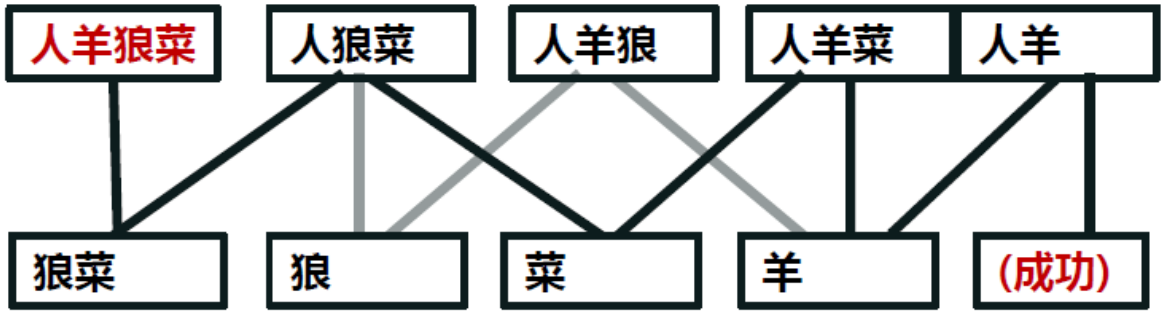
队列的应用

- 只要满足先来先服务特性的应用均可采用队列作为其数据组织方式或中间数据结构。
- 调度或缓冲
  - 消息缓冲器
  - 邮件缓冲器
  - 计算机的硬设备之间的通信也需要队列作为数据缓冲
  - 操作系统的资源管理
- 宽度优先搜索

队列的应用—农夫过河问题

- 农夫过河问题
  - 只有人能撑船，船上只有两个位置（包括人）
  - 狼羊、羊菜不能在没人时共处





数据抽象

- 每个角色的位置进行描述
  - 农夫、狼、菜和羊，四个目标各用一位（bit）
    （假定按照农夫、狼、白菜、羊次序），目标在起始岸位置：0，目标岸：1

0	0	0	0

  - 如0101 表示农夫、白菜在起始岸，而狼、羊在目标岸（此状态为不安全状态）

0	1	0	1

数据的表示

- 用整数status 表示上述四位二进制描述的状态
  - 整数0x08 表示的状态
  - 整数0x0F 表示的状态

1	0	0	0

1	1	1	1

- 如何从上述状态中得到每个角色所在的位置?
  - 设计一个状态判断函数
  - 函数返回值为真（1），表示所考察人或物在目标岸
  - 否则，表示所考察人或物在起始岸

角色位置判断函数

```
bool farmer(int status)
{  return ((status & 0x08)  != 0);     }

bool wolf(int status)
{  return ((status & 0x04)  != 0);     }

bool cabbage(int status)
{  return ((status & 0x02)  != 0);     }

bool goat(int status)
{  return ((status & 0x01)  != 0);     }
```

人	狼	菜	羊
1	x	x	x
x	1	x	x
x	x	1	x
x	x	x	1

安全状态判断

```
① bool safe(int status)                    // 返回 true:安全，false:不安全
② {
③     if ((goat(status) == cabbage(status)) &&  (goat(status) != farmer(status)))
④             return(false);               // 羊吃白菜
⑤     if ((goat(status) == wolf(status)) &&  (goat(status) != farmer(status)))
⑥             return(false);               // 狼吃羊
⑦     return(true);                                  // 其它状态为安全
⑧ }
```

人	狼	菜	羊
0	1	0	1

算法抽象

- 问题变为：
  - 从状态0000（整数0）出发，
  - 寻找全部由安全状态构成的状态序列，
  - 以状态1111（整数15）为最终目标。
  - 状态序列中每个状态都可以从前一状态通过农夫（可以带一样东西）划船过河的动作到达。
  - 序列中不能出现重复状态

算法分析

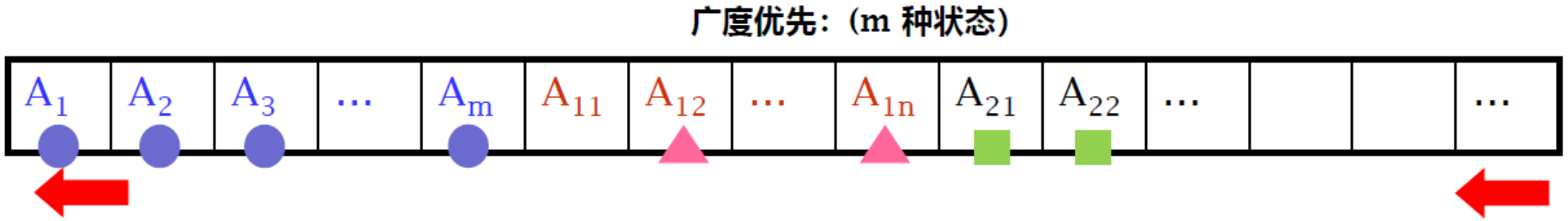
- 该问题的求解可以使用试探法，每一步都搜索当前状态下所有可能的选择，对合适的选择再考虑下一步的各种方案。
- 两种不同的搜索策略：

  广度优先搜索：搜索该步的所有可能状态，再进一步考虑后面的各种情况；（队列应用）

  深度优先搜索：沿某一状态走下去，不行再回头。（栈应用）

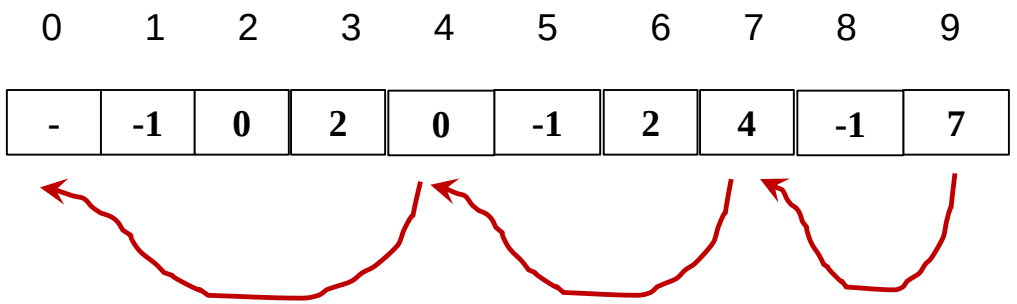
广度优先搜索算法分析

- 采用队列做辅助结构，把本步可以到达的所有状态都放在队列中
- 从队列中顺序取出状态，对其处理，处理过程中再把下一步可以到达的状态放在队列中
- 由于队列的操作按照先进先出原则，因此只有前一步的所有情况都处理完后才能进入下一步。



算法设计

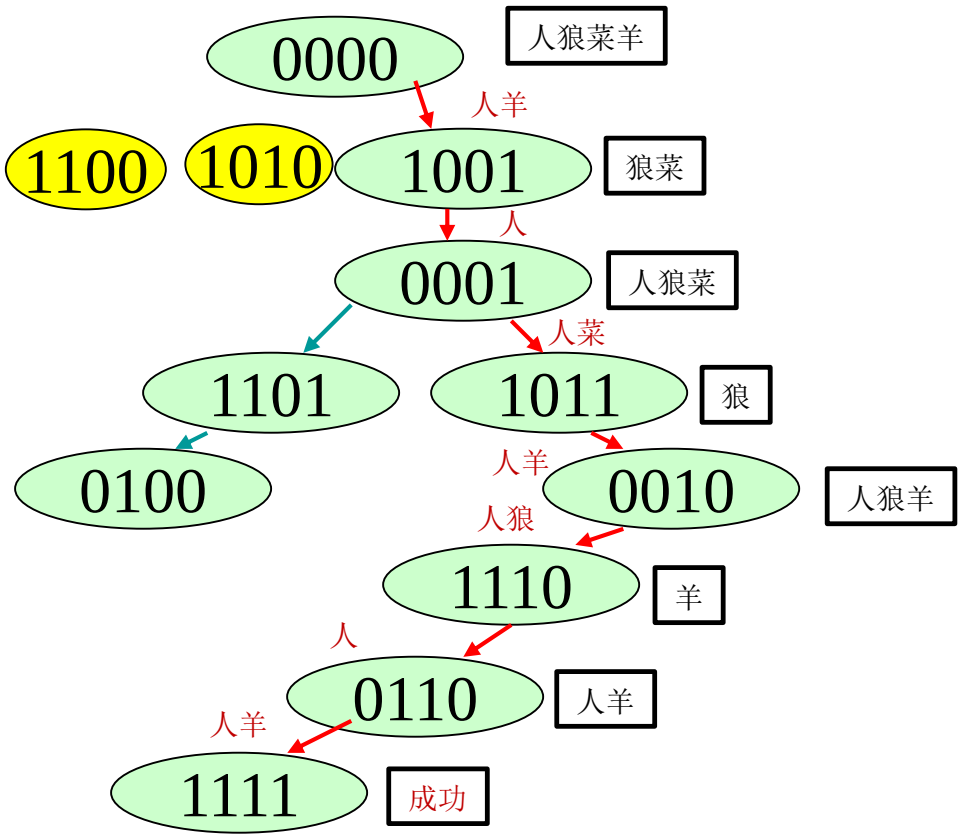
- 还需要定义一个数据结构，记录已被访问过的各个状态，以及已被发现的能够到达当前这个状态的路径
  - 用顺序表route 的第i个元素记录状态i是否已被访问过
  - 若route[i] 已被访问过，则记入前驱状态值；-1表示未被访问
  - route 的大小（长度）为16



算法实现

- moveTo整数队列，元素为可以安全到达的中间状态。
- route顺序表，记录各个状态被访问过的情况（大小为16数组）：
  - -1表示未被访问，否则记录前驱状态值的下标。
  - 最后，（route[15]为非负值时）利用route建立正确的状态路径。
- 计算结果

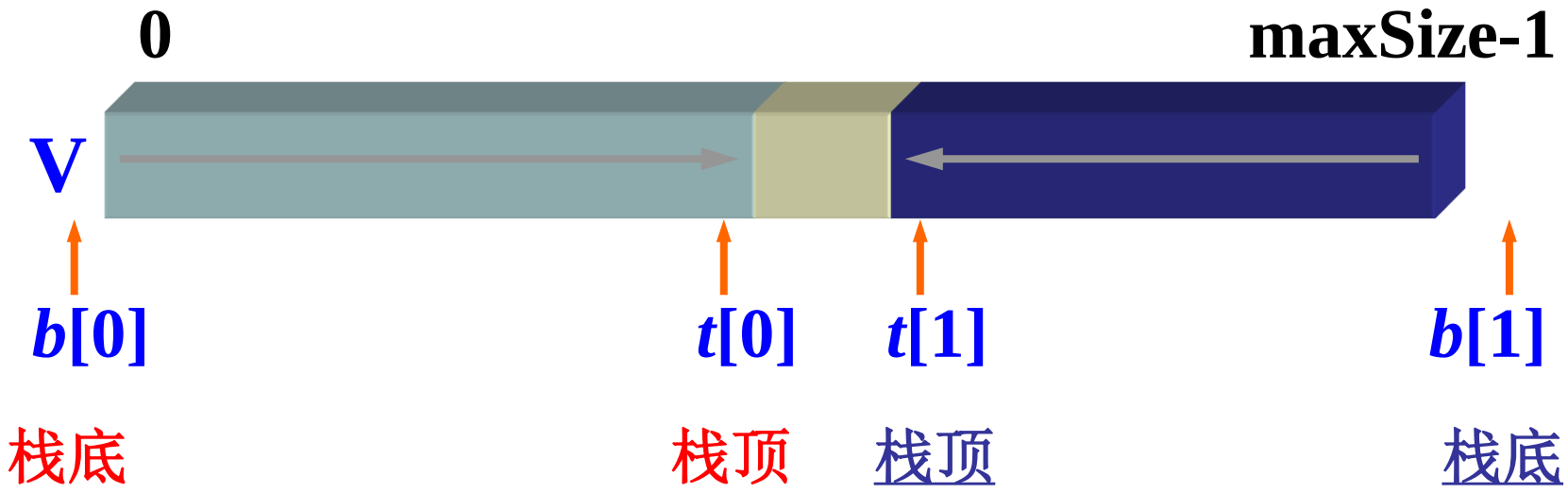
  route顺序表:15, 6, 14, 2, 11, 1, 9, 0



顺序队列与链式队列的比较

- 变种的栈或队列结构
  - 双端队列：在队首和队尾都可以插入、删除的队列
  - 双栈：两个底部相连的栈，共享一块数据空间
  - 超队列：是一种被限制的双端队列，删除操作只允许在一端进行，插入操作却可以在两端同时进行
  - 超栈：是一种插入受限的双端队列，即插入限制在一端而删除仍允许在两端进行。

双栈共享一个栈空间





思考题：栈和队列的互模拟

- 如何用两个栈模拟一个队列?
- 如何用两个队列模拟一个栈?

练习

- 将1，2，3，……，n,顺序入栈，其输出序列为p1，p2，p3,……,pn，若p1=n，则pi为__？ n-i+1

- 假设以数组A[m]存放循环队列的元素，其头指针是front，当前队列有k个元素，则队列的尾指针为_____ (front+k)%m

- 设栈S和队列Q的初始状态为空，元素a、b、c、d、e、f依次通过栈S，一个元素出栈后即进入队列Q。若这6个元素出队列的顺序是b、d、c、f、e、a，请写出它们进栈出栈的顺序。 a b (b) c d (d) (c) e f (f) (e) (a)



补充内容：运算符与表达式

- C语言中的运算符（Operator）规定了对操作数的处理规则：
  - 算术运算符：+，-，*，/，%，++，--
  - 关系运算符：>，<，>=，<=，==，!=
  - 逻辑运算符：!，&&，||
  - 位运算符：>>，<<，&，|，^，~
  - 赋值运算符：=，及其扩展赋值运算符如+=，-=，*=，/=等
  - 条件运算符：？：



逻辑运算表达式

使用 逻辑运算符号，

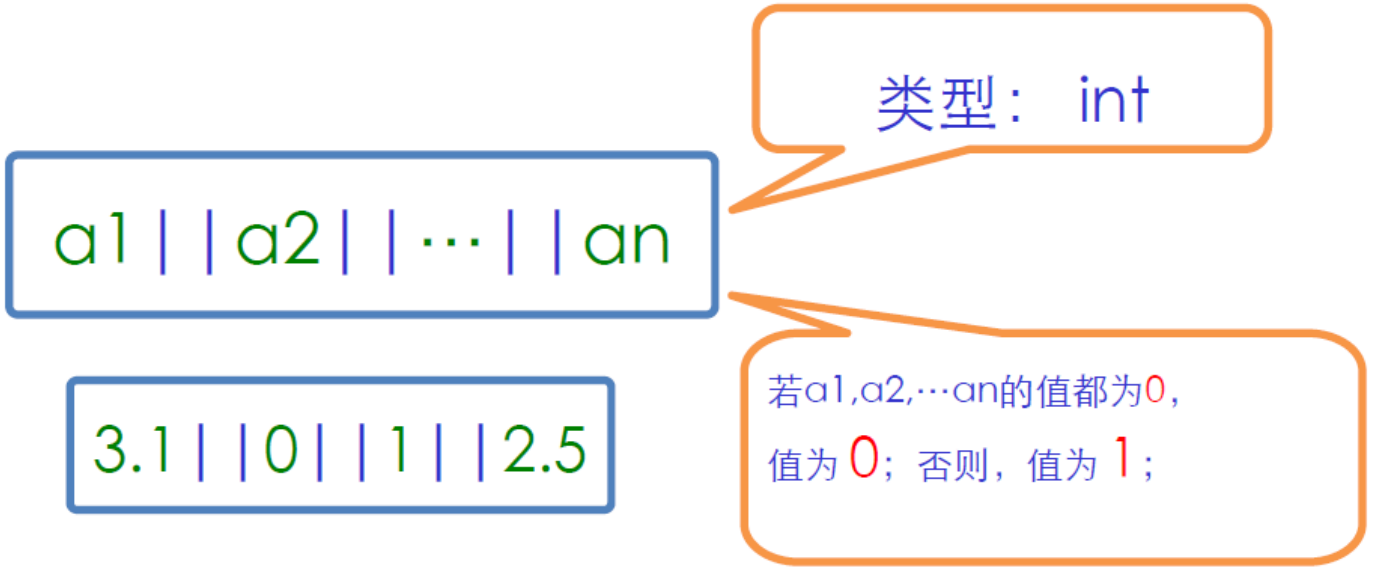
按照一定的规则，将操作数连接起来的表达式，称为 逻辑运算表达式

关于 操作数：

任何类型为int, float, 或 double的表达式，都可以作为逻辑运算表达式的操作数

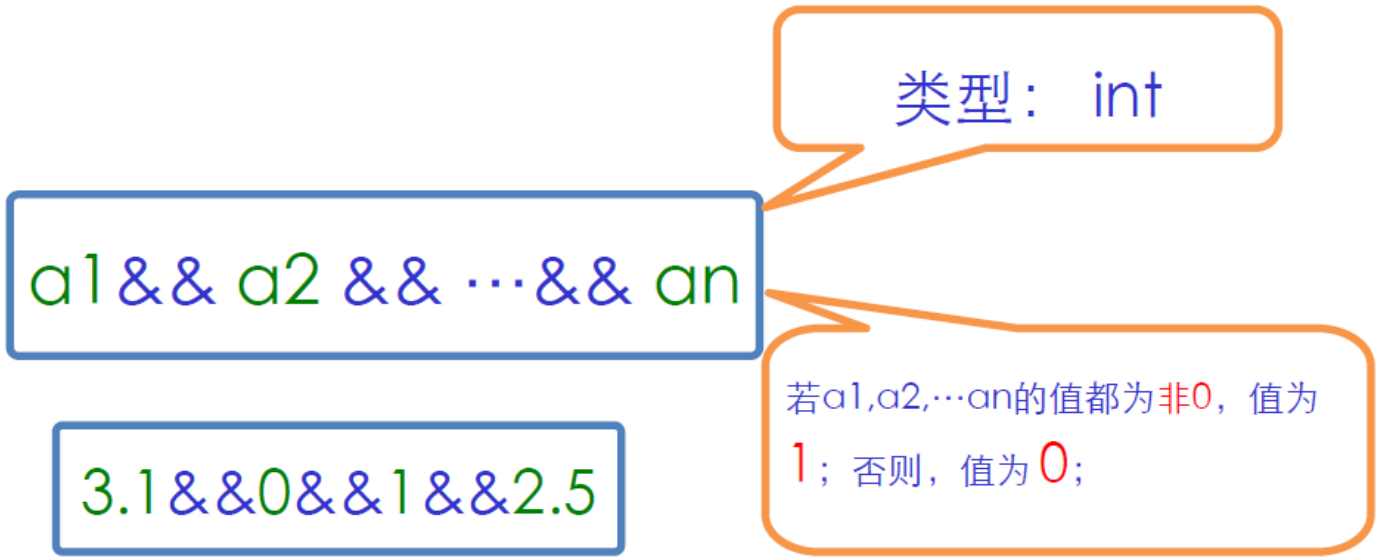
逻辑运算符号 逻辑或：||

- 假设a1，a2，…an是类型为int，float，或double的表达式



逻辑运算符号 逻辑与：&&

- 假设a1，a2，…an是类型为int，float，或double的表达式





位运算表达式

使用 位运算符，

按照一定的规则，将操作数连接起来的表达式，称为 位运算表达式

关于 操作数：

必须是整型数据！



位运算表达式

- &(按位与)
- |（按位或）
- ^（按位异或）
- ~（按位非）
- <<(左位移)：将左侧操作数的二进制数值向左移动若干位（由右侧的操作数给出），移出去的位丢弃，空出的位用0填补
- >>(右位移)：将左侧操作数的二进制数值向右移动若干位（由右侧的操作数给出），移出去的位丢弃，空出的位用符号位（对有符号数）或0（对无符号数）来填补

例：异或

```
  1 1 0 0 0 1
^ 0 1 1 1 1 1
-------------
  1 0 1 1 1 0
```

C=A^B
A=C^B
B=C^A

- 设计一个算法来实现字符串逆序存储，要求不另设串存储空间；

  利用字符串的结束符'\0'，作为中间变量

  利用a=a+b;b=a-b;a=a-b;

  利用a=a^b;b=a^b;a=a^b;（异或）



位运算符号

- 无符号数：没有符号位，不管左移或右移，空出的位用0填补。

- 位移运算的实质（在不发生溢出时）
  - 左移：x<<n，相当于x* 2^n
  - 右移：x>>n，相当于x/2^n

位运算符号

- 有符号数：做位移运算时，符号位不参与移动
  - 左移时，空出的位用0填补；
  - 右移时，空出的位用符号位填补。（数据的补码表示）





数据结构与算法